Tarea 2.1. Contenedores LXC — Guía de práctica

Entorno y criterio de documentación

Realizo la práctica en mi host `Sergio-PC`, con Debian 13 (Trixie), LXC 6.0.4 y NetworkManager. Utilizo LXC clásico y contenedores privilegiados, gestionados con `sudo` desde el host. Dentro de los contenedores trabajo como root mediante `lxc-attach`, salvo al comprobar SSH con el usuario sin privilegios `alumno`.

Organizo esta guía a partir de mis salidas de terminal y del PDF de clase. Elimino intentos fallidos, avisos de instalación irrelevantes, configuraciones duplicadas y listados extensos de imágenes. Los bloques de configuración representan la configuración limpia que debo conservar; no todos son volcados literales de un único instante de la práctica. Las salidas de comprobación son extractos de los resultados registrados.

1. Primeros pasos con LXC

1.1. Instalación y comprobación del host

En el host instalo LXC y los paquetes utilizados en clase:

```
sudo apt install -y lxc apparmor lxcfs
```

Con `lxc` dispongo de las herramientas de gestión; con AppArmor dispongo del confinamiento que emplea el perfil generado, y con LXCFS puedo consultar desde el contenedor los recursos que le he limitado.

Mi sistema confirma las versiones instaladas:

```
lxc ya está en su versión más reciente (1:6.0.4-4+deb13u4).
apparmor ya está en su versión más reciente (4.1.0-1).
lxcfs ya está en su versión más reciente (6.0.4-1+deb13u1).
```

Compruebo las capacidades del kernel:

```
lxc-checkconfig
```

Entre las comprobaciones obtengo:

```
LXC version 6.0.4
Cgroups: enabled
Cgroup v2 mount points:
 - /sys/fs/cgroup
Cgroup memory controller: enabled
Cgroup cpuset: enabled
Bridges: enabled, loaded
```

Compruebo el servicio de red y el bridge de LXC:

```
systemctl status lxc-net --no-pager
ip -4 addr show lxcbr0
cat /etc/lxc/default.conf
```

El servicio aparece activo y `lxcbr0` tiene `10.0.3.1/24`. La configuración inicial de red es:

```
lxc.net.0.type = veth
lxc.net.0.link = lxcbr0
lxc.net.0.flags = up
```

Con `veth` conecto el contenedor mediante un par de interfaces virtuales; `link` identifica el bridge del host y `flags = up` levanta el enlace.

1.2. Creación del contenedor Ubuntu

Creo el contenedor con el asistente interactivo:

```
sudo lxc-create -n contenedor1 -t download
```

Con `-n` asigno el nombre y con `-t download` descargo una imagen. En la lista disponible selecciono:

```
Distribution: ubuntu
Release: resolute
Architecture: amd64
```

La creación termina con:

```
You just created an Ubuntu resolute amd64 (20261013_07:42) container.
```

Listo los contenedores:

```
sudo lxc-ls -f
```

```
NAME        STATE   AUTOSTART GROUPS IPV4 IPV6 UNPRIVILEGED
contenedor1 STOPPED 0         -      -    -    false
contenedor2 STOPPED 0         -      -    -    false
```

Mantengo `contenedor2` en las evidencias porque aparece en mi entorno, aunque no lo utilizo para el escenario.

1.3. Arranque, direccionamiento y conectividad

Arranco el contenedor y consulto su estado e IP:

```
sudo lxc-start -n contenedor1
sudo lxc-info -n contenedor1
sudo lxc-attach -n contenedor1 -- ip -4 addr show
sudo lxc-attach -n contenedor1 -- ip route
```

`lxc-start` arranca el sistema y `lxc-info` muestra información desde el host. Con `lxc-attach ... -- comando` ejecuto una orden dentro sin abrir una sesión interactiva.

En el primer arranque registro:

```
inet 10.0.3.156/24 metric 100 brd 10.0.3.255 scope global dynamic eth0
default via 10.0.3.1 dev eth0 proto dhcp src 10.0.3.156 metric 100
```

Compruebo que dispongo de ping y pruebo salida por IP y por nombre:

```
sudo lxc-attach -n contenedor1 -- apt install -y iputils-ping
sudo lxc-attach -n contenedor1 -- ping -c 3 8.8.8.8
sudo lxc-attach -n contenedor1 -- ping -c 3 google.com
```

En ambas pruebas obtengo tres respuestas y un 0 % de pérdida. La primera comprueba conectividad IP y la segunda también comprueba resolución DNS.

1.4. Actualización sin entrar en la consola

Ejecuto directamente desde el host:

```
sudo lxc-attach -n contenedor1 -- apt update
```

Con esta orden actualizo el índice de paquetes de Ubuntu sin abrir una consola interactiva del contenedor. Registro acceso correcto a los repositorios `resolute`, `resolute-security` y `resolute-updates`.

1.5. Límite de memoria y CPU

Edito el fichero del contenedor:

```
sudo nano /var/lib/lxc/contenedor1/config
```

Añado:

```
lxc.cgroup2.memory.max = 512M
lxc.cgroup2.cpuset.cpus = 0
```

Con la primera línea limito la memoria a 512 MiB; con la segunda restrinjo la ejecución a la CPU lógica 0. No asigno una CPU virtual nueva: limito las CPU del host que puede utilizar el contenedor. Este ajuste no limita por sí solo la swap.

Reinicio para aplicar la configuración:

```
sudo lxc-stop -n contenedor1
sudo lxc-start -n contenedor1
```

Compruebo los límites desde dentro:

```
sudo lxc-attach -n contenedor1 -- free -h
sudo lxc-attach -n contenedor1 -- cat /proc/cpuinfo
```

```
               total        used        free      shared  buff/cache   available
Mem:           512Mi        15Mi       495Mi       120Ki       836Ki       496Mi
Swap:           14Gi          0B        14Gi
```

En `/proc/cpuinfo` aparece un único bloque `processor : 0`. Los campos descriptivos del procesador físico no equivalen al número de CPU permitidas al contenedor.

2. Cambio de red del contenedor

2.1. Situación inicial

Antes del cambio registro esta configuración:

```
lxc.net.0.type = veth
lxc.net.0.link = lxcbr0
lxc.net.0.flags = up
```

Tras los reinicios, la IP DHCP registrada es `10.0.3.29/24` y la puerta de enlace sigue siendo `10.0.3.1`. No interpreto que esta IP distinta sea un fallo: registro las direcciones observadas en cada momento.

2.2. Conexión a default

En mi práctica utilizo la red `default` de libvirt y su bridge `virbr0`, en lugar de `br-nat`. Para el apartado 2 documento esta elección como adaptación respecto al texto que menciona `red-nat`; el apartado 3 sí permite expresamente `default`.

Consulto las redes y bridges:

```
sudo virsh net-list --all
ip -br link show type bridge
```

Paro el contenedor y cambio únicamente su conexión de red:

```
sudo lxc-stop -n contenedor1
sudo sed -i 's/^lxc.net.0.link = lxcbr0/lxc.net.0.link = virbr0/'
/var/lib/lxc/contenedor1/config
sudo grep '^lxc.net' /var/lib/lxc/contenedor1/config
sudo lxc-start -n contenedor1
```

Con `sed -i` sustituyo la línea del bridge dentro del fichero. Verifico el resultado antes de arrancar:

```
lxc.net.0.type = veth
lxc.net.0.link = virbr0
lxc.net.0.flags = up
```

2.3. Comprobación

```
sudo lxc-attach -n contenedor1 -- ip -4 addr show
sudo lxc-attach -n contenedor1 -- ping -c 3 8.8.8.8
```

Registro:

```
inet 192.168.122.179/24 metric 100 brd 192.168.122.255 scope global dynamic eth0

3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 14.683/14.875/15.043/0.148 ms
```

Con ello compruebo la nueva IP y la salida al exterior. Si consulto `lxc-ls` inmediatamente después del arranque, DHCP puede no haber terminado; por eso utilizo la comprobación posterior que ya muestra dirección.

3. Escenario router/NAT con almacenamiento compartido

3.1. Diseño

Monto este escenario:

```
Internet
   |
Host / NAT de libvirt — virbr0 (192.168.122.0/24)
   |
router eth0: 192.168.122.218/24 (DHCP)
router eth1: 10.10.10.1/24
   |
br-contenedores (sin IP en el host)
   |
servidorweb eth0: 10.10.10.10/24
Gateway: 10.10.10.1
```

Mi servidor web solo tiene una interfaz en la red interna. Para salir utiliza el router, que a su vez sale mediante la red NAT de libvirt.

3.2. Creación de contenedores

Desde el host ejecuto:

```
sudo lxc-create -n router -t download -- -d debian -r bookworm -a amd64
sudo lxc-create -n servidorweb -t download -- -d ubuntu -r noble -a amd64
```

Con `-d` selecciono distribución, con `-r` versión y con `-a` arquitectura. Utilizo Debian Bookworm para el router y Ubuntu 24.04 Noble para el servidor web.

3.3. Bridge persistente con NetworkManager

Como mi host utiliza NetworkManager, creo el bridge con `nmcli`, sin utilizar `virsh` para crearlo:

```
sudo nmcli connection add type bridge ifname br-contenedores con-name br-
contenedores ipv4.method disabled ipv6.method ignore
sudo nmcli connection up br-contenedores
```

Creo un perfil persistente y lo activo. Desactivo la configuración IPv4 del host en este bridge y dejo IPv6 en modo `ignore`, sin configurar una red IPv6 para esta práctica. No conecto mi interfaz Wi-Fi al bridge.

Compruebo el perfil:

```
nmcli connection show br-contenedores | grep -E
'connection.autoconnect|ipv4.method|ipv6.method'
ip a show br-contenedores
```

```
connection.autoconnect:                 sí
ipv4.method:                            disabled
ipv6.method:                            ignore
```

Al crearlo aparece `NO-CARRIER` porque aún no tiene interfaces de contenedores conectadas. El indicador administrativo `UP` confirma que lo he levantado.

3.4. Conexiones y arranque automático

Edito desde el host:

```
sudo nano /var/lib/lxc/router/config
sudo nano /var/lib/lxc/servidorweb/config
```

Conservo las líneas generales de cada plantilla, sustituyo los bloques de red anteriores y evito duplicarlos.

En `router` dejo:

```
lxc.start.auto = 1
lxc.start.delay = 5

lxc.net.0.type = veth
lxc.net.0.link = virbr0
lxc.net.0.flags = up
lxc.net.0.name = eth0

lxc.net.1.type = veth
lxc.net.1.link = br-contenedores
lxc.net.1.flags = up
lxc.net.1.name = eth1
```

En `servidorweb` dejo:

```
lxc.start.auto = 1
lxc.start.delay = 5

lxc.net.0.type = veth
lxc.net.0.link = br-contenedores
lxc.net.0.flags = up
lxc.net.0.name = eth0
```

Con `start.auto` solicito el arranque automático y con `start.delay` añado una espera de cinco segundos después de iniciar cada contenedor. Esta espera no garantiza por sí sola el orden de creación del bridge durante el arranque del host.

Arranco los dos contenedores:

```
sudo lxc-start -n router
sudo lxc-start -n servidorweb
```

3.5. Red dentro de router

Entro como root:

```
sudo lxc-attach -n router
mkdir -p /etc/systemd/network
vi /etc/systemd/network/10-eth1.network
```

Con `vi` pulso `i` para editar y guardo con `Esc`, `:wq` y Enter. Escribo:

```
[Match]
Name=eth1

[Network]
Address=10.10.10.1/24
```

Asigno la IP interna de forma persistente con systemd-networkd y mantengo la configuración DHCP de la interfaz externa de la plantilla.

```
systemctl restart systemd-networkd
ip -4 addr show eth1
ip route
exit
```

Reinicio el gestor para aplicar el fichero, consulto dirección y rutas y vuelvo al host.

Registro estas rutas:

```
default via 192.168.122.1 dev eth0 proto dhcp src 192.168.122.218 metric 1024
10.10.10.0/24 dev eth1 proto kernel scope link src 10.10.10.1
192.168.122.0/24 dev eth0 proto kernel scope link src 192.168.122.218
192.168.122.1 dev eth0 proto dhcp scope link src 192.168.122.218 metric 1024
```

3.6. Red dentro de servidorweb

Entro y edito Netplan:

```
sudo lxc-attach -n servidorweb
mkdir -p /etc/netplan
vi /etc/netplan/10-lxc.yaml
```

Dejo la configuración final, incluidos los DNS que añadí para resolver nombres:

```
network:
  version: 2
  ethernets:
    eth0:
      addresses: [10.10.10.10/24]
      routes:
        - to: default
          via: 10.10.10.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Asigno IP estática, puerta de enlace y servidores DNS. Protejo el fichero:

```
chmod 600 /etc/netplan/10-lxc.yaml
```

Para esta imagen LXC aplico la solución del PDF que simula `udevadm`:

```
mkdir -p /usr/local/bin
printf '#!/bin/sh\nexit 0\n' > /usr/local/bin/udevadm
chmod +x /usr/local/bin/udevadm
netplan apply
```

El pequeño script termina sin error y permite aplicar Netplan en el entorno descrito en clase. No instala udev ni reproduce sus funciones; lo utilizo únicamente como adaptación de esta práctica.

```
ip -4 addr show eth0
ip route
exit
```

Obtengo `10.10.10.10/24` y ruta por defecto vía `10.10.10.1`.

3.7. Reenvío y NAT del router

Entro en el router e instalo iptables:

```
sudo lxc-attach -n router
apt update
apt install -y iptables
```

Activo el reenvío IPv4 de forma persistente:

```
echo 'net.ipv4.ip_forward=1' > /etc/sysctl.d/99-forward.conf
sysctl --system
```

La salida confirma `net.ipv4.ip_forward = 1`. Este parámetro permite al sistema encaminar tráfico entre interfaces; no sustituye a la política del cortafuegos.

Permito el reenvío por defecto y creo el NAT de salida:

```
iptables -P FORWARD ACCEPT
iptables -t nat -A POSTROUTING -s 10.10.10.0/24 -o eth0 -j MASQUERADE
```

Elijo una política permisiva para este laboratorio, sin reglas adicionales de filtrado. La regla de `POSTROUTING` traduce el origen del tráfico de la red interna al salir por `eth0`. Utilizo `MASQUERADE` porque la IP externa se obtiene por DHCP; no necesito que cambie en cada arranque para que esta elección tenga sentido.

Guardo las reglas:

```
apt install -y iptables-persistent
netfilter-persistent save
cat /etc/iptables/rules.v4
```

El fichero contiene `:FORWARD ACCEPT [0:0]` y la regla `MASQUERADE`. Esta es la sintaxis de `iptables-save`; no aparece literalmente `-P FORWARD ACCEPT` dentro del fichero guardado.

```
exit
sudo lxc-attach -n servidorweb -- ping -c 3 8.8.8.8
sudo lxc-attach -n servidorweb -- ping -c 3 google.com
```

Tras configurar DNS, ambas pruebas terminan con tres respuestas y 0 % de pérdida.

3.8. SSH con usuario sin privilegios y salto

Instalo el servidor SSH en ambos contenedores desde el host:

```
sudo lxc-attach -n router -- bash -c 'apt update && apt install -y openssh-server'
sudo lxc-attach -n servidorweb -- bash -c 'apt update && apt install -y openssh-server'
```

Creo el usuario sin privilegios. En la guía normalizo el campo informativo `--gecos` a vacío: no es una contraseña.

```
sudo lxc-attach -n router -- adduser --disabled-password --gecos "" alumno
sudo lxc-attach -n servidorweb -- adduser --disabled-password --gecos "" alumno
```

No añado el usuario al grupo sudo. Como lo creo sin contraseña, asigno una para el primer acceso y la instalación de claves:

```
sudo lxc-attach -n router -- passwd alumno
sudo lxc-attach -n servidorweb -- passwd alumno
```

Introduzco la contraseña cuando se solicita, sin incluirla en el documento.

Reutilizo la clave RSA que ya tengo en el host y copio su parte pública al router:

```
ssh-copy-id -i ~/.ssh/id_rsa.pub alumno@192.168.122.218
ssh alumno@192.168.122.218
exit
```

La salida confirma `Number of key(s) added: 1`. Para instalar la misma clave pública en el servidor web, ejecuto desde el host:

```
ssh -J alumno@192.168.122.218 alumno@10.10.10.10 'mkdir -p ~/.ssh && chmod 700
~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys' <
~/.ssh/id_rsa.pub
```

Con `-J` atravieso el router. La redirección `<` lee la clave pública en mi host y el comando remoto la añade al fichero del servidor web. La clave privada permanece en mi host; no la copio al router ni necesito generar una clave allí.

Compruebo el acceso final:

```
ssh -J alumno@192.168.122.218 alumno@10.10.10.10
```

```
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.12.111+deb13-amd64 x86_64)
alumno@servidorweb:~$
```

Salgo con `exit` para continuar administrando desde el host.

3.9. Servidor Nginx y directorio compartido

Instalo Nginx y compruebo su servicio:

```
sudo lxc-attach -n servidorweb -- bash -c 'apt update && apt install -y nginx'
sudo lxc-attach -n servidorweb -- systemctl status nginx --no-pager
```

Registro el servicio habilitado y `Active: active (running)`.

Creo la página en el host:

```
sudo mkdir -p /opt/web
echo '<h1>Hola desde el host y LXC</h1>' | sudo tee /opt/web/index.html
```

Con `tee` escribo el contenido con permisos de administrador. Añado en `/var/lib/lxc/servidorweb/config`:

```
lxc.mount.entry = /opt/web var/www/html none bind 0 0
```

Realizo un montaje bind: el contenedor accede al directorio del host, no a una copia. El destino `var/www/html` es relativo a la raíz del contenedor y no lleva barra inicial. Instalo Nginx antes, por lo que el directorio destino ya existe.

```
sudo lxc-stop -n servidorweb
sudo lxc-start -n servidorweb
sudo lxc-attach -n servidorweb -- ls -l /var/www/html
sudo lxc-attach -n servidorweb -- cat /var/www/html/index.html
```

Registro:

```
total 4
-rw-r--r-- 1 root root 34 Oct 15 16:15 index.html
<h1>Hola desde el host y LXC</h1>
```

3.10. Publicación web mediante DNAT

Entro en el router y redirijo el puerto TCP 80 de la interfaz externa al servidor web:

```
sudo lxc-attach -n router
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT --to-destination
10.10.10.10:80
netfilter-persistent save
iptables -t nat -L PREROUTING -n -v
exit
```

Con DNAT cambio el destino de las peticiones entrantes hacia `10.10.10.10:80`. Mantengo `FORWARD ACCEPT`, por lo que no añado reglas de permiso específicas. Guardo de nuevo para incluir la nueva redirección.

3.11. Comprobación desde el host y modificación inmediata

Desde el host consulto la IP externa del router:

```
curl http://192.168.122.218/
```

```
<h1>Hola desde el host y LXC</h1>
```

Modifico únicamente el fichero del host y repito la consulta:

```
echo '<h1>Cambio hecho desde el host</h1>' | sudo tee /opt/web/index.html
curl http://192.168.122.218/
```

```
<h1>Cambio hecho desde el host</h1>
```

Compruebo así que DNAT lleva la petición al servidor web y que el montaje comparte el contenido sin copiar archivos ni reiniciar Nginx.

4. Alcance de las comprobaciones y evidencias pendientes

He registrado el funcionamiento del escenario, los perfiles persistentes y las reglas guardadas. No incluyo como realizada una prueba de reinicio completo del host, porque no aparece en las salidas aportadas. El arranque automático de LXC necesita que los bridges estén disponibles cuando arranquen los contenedores; si verifico un reinicio, comprobaré también ese orden.

La IP externa del router es DHCP. En esta ejecución utilizo `192.168.122.218`; si cambia, actualizaré los comandos de SSH y la dirección de acceso web.

Para completar literalmente la entrega, me falta insertar una captura real del acceso web desde el host. Dispongo de las salidas de `curl`, pero no de una imagen adjunta. Puedo capturar la terminal con las dos consultas o el navegador abierto en `http://192.168.122.218/`, según el formato solicitado por el profesor. No sustituyo esa captura por una imagen inventada.

Base documental

He elaborado la guía con mi registro de terminal `paste.txt`, las comprobaciones de la conversación y el PDF de clase `lxc.pdf`. No he reproducido claves públicas completas,

contraseñas ni mensajes sin relación con los objetivos de la tarea.